QUEST

Qualitative Untersuchung und Evaluation von Sprachmodellen Tool

Technical Documentation – Version 0.1.0

Authors:

- Felix Gratzkowski
- Dr. rer. nat. Sebastian Bader
- Dr.-Ing. Robin Nicolay

University of Rostock
Institute for Visual & Analytic Computing – 2024
Mobile Multimedia Information Systems

Part of the KI-Med Collaboration Platform (KiMeKo)
Funded by BMBF (01IS24056D)

Table of Contents

1. Executive Summary

The QUEST (Qualitative Untersuchung und Evaluation von Sprachmodellen Tool) project aims to provide a comprehensive platform for interactive testing, comparison, and evaluation of multiple language models. This report details the design, implementation, and evaluation of the QUEST platform, focusing on its architecture, features, and performance characteristics.

2. Introduction

QUEST was developed to address the need for a tool to evaluate and compare the performance of various language models. The platform supports real-time interactions, model comparisons, and detailed evaluations, making it an essential tool for researchers and developers working with language models.

3. System Overview

3.1 High-Level Architecture

The QUEST platform follows a client-server architecture, with a React-based frontend and a Node.js backend. The client communicates with the server using Socket.IO for real-time, bidirectional communication. The server connects to multiple language model providers using a modular adapter pattern.

3.2 Technology Stack

Frontend Technologies

- **React 18**: Component-based architecture for efficient UI rendering
- **TypeScript**: Static typing for improved code reliability and developer experience
- **Vite**: Fast build tooling and development server with hot module replacement
- **Mantine UI**: Modern React component library for consistent design language
- **Socket.IO Client**: Real-time bidirectional communication
- **ApexCharts**: Data visualization for model comparison metrics

Backend Technologies

- **Node.js**: JavaScript runtime for server-side operations
- **Express.js**: Web application framework handling HTTP requests
- **Socket.IO Server**: WebSocket implementation for bidirectional communication
- **bcrypt-ts**: Password hashing and authentication
- **jsonwebtoken**: JWT-based authentication system
- **LLM Provider SDKs**: OpenAI and Ollama integration libraries

DevOps & Deployment

- **Docker & Docker Compose**: Containerization for consistent deployment
- **Nginx**: Reverse proxy for HTTPS and load balancing
- **ESLint & Prettier**: Code quality and formatting

3.3 Key Functionalities

- **Multi-model comparison**: Side-by-side interaction with multiple language models
- **Real-time streaming**: Instant model responses and streaming chunks of LLM output
- **Evaluation metrics**: Cosine similarity, Levenshtein distance, and Jaccard similarity
- **Project management**: Save/load functionality with JSON-based project format

4. Design Choices

4.1 Client-Side Design

The client application uses a hierarchical component structure with custom hooks for state management. Key components include:

- **App.tsx**: Root component orchestrating application flow
- **PromptWindow**: User input and prompt management
- **EvaluationWindow**: Visualization of evaluation metrics
- **ChatWindow**: Real-time chat interface with language models

4.2 Server-Side Design

The server follows a modular design pattern with clear separation of concerns. Key modules include:

- **SocketAPIHandler**: Routes events to appropriate handlers
- **LLMAdapterManager**: Manages model adapters and forwards requests

4.3 Data Flow & Communication

The QUEST application uses Socket.IO for real-time bidirectional communication between the client and server. This socket system enables features like instant model responses, streaming chunks of LLM output as they're generated, and aborting ongoing requests.

5. Implementation Details

Adapter Pattern for LLM Providers

QUEST implements a flexible adapter pattern allowing connection to multiple language model providers. Each adapter extends a base class and implements provider-specific logic.

Socket-Based Communication Protocol

All communication follows a standardized message format defined in the `SocketMessage` interface. The socket system uses a predefined set of events in the `EVENTS` enum to handle various operations.

Key Implementation Challenges

1. **Handling Concurrent LLM Requests**: Implemented a robust queue system to manage multiple simultaneous requests.
2. **Stream Processing of LLM Responses**: Developed real-time streaming responses while maintaining message ordering.
3. **Synchronizing Multiple Chat Sessions**: Ensured consistent chat state across different models.
4. **Evaluation System Implementation**: Created a fair comparison methodology for different model responses.

Optimization Techniques

1. **Request Queueing and Prioritization**: Limits concurrent requests per adapter based on configuration.
2. **Memoization for UI Performance**: Prevents unnecessary re-renders.
3. **Efficient DOM Updates**: CSS optimizations and conditional rendering.

Scalability Considerations

1. **Horizontal Scaling Support**: Stateless server design and Docker containerization.
2. **LLM Provider Load Management**: Configurable request limits and timeouts.
3. **Resource Efficiency**: Clean request cancellation and memory management.
4. **Future Extensibility**: Modular adapter system and configurable evaluation metrics.

6. Outcome & Goal Evaluation

Original Objectives Assessment

QUEST set out to create a comprehensive platform for interactive testing and comparative evaluation of multiple language models. The final implementation successfully achieved these objectives.

Core Objectives Achievement

Objective	Achievement Status	Implementation Evidence
Multi-model comparison	✅ Fully achieved	Implemented TabList component for side-by-side model interaction
Response evaluation metrics	✅ Fully achieved	Robust evaluation system with multiple similarity metrics
Support for different LLM providers	✅ Fully achieved	Modular adapter system supporting both cloud and local models
Interactive visualization	✅ Fully achieved	Heatmap visualizations and detailed comparisons
Project management	✅ Fully achieved	Comprehensive save/load functionality
Real-time streaming responses	✅ Fully achieved	WebSocket-based streaming implementation

Lessons Learned

1. **WebSocket Optimization**: Challenges in managing connection stability and message ordering.
2. **Request Queue Management**: Essential for managing concurrent LLM requests.
3. **Component Design**: Facilitated independent development of features.
4. **Performance Challenges**: CSS optimizations were necessary for smooth performance.

Future Improvement Roadmap

1. **Enhanced Error Handling**: More comprehensive system with detailed error categorization.
2. **Expanded Model Parameters**: Support for adjustable generation parameters.
3. **Extended Evaluation Metrics**: Additional metrics focused on specific attributes.
4. **Collaborative Features**: Real-time collaboration and shared projects.
5. **Automated Analysis**: AI-powered analysis of model strengths and weaknesses.
6. **Benchmark Suite**: Standardized test suites for comprehensive model evaluation.
7. **Extended Visualization**: More sophisticated visualization techniques.

7. Conclusion

QUEST has successfully achieved its primary objectives of creating a flexible, powerful platform for comparative LLM analysis. The adapter-based architecture provides an excellent foundation for future expansion, while the evaluation system offers valuable insights into model performance differences. The implementation strikes a good balance between feature richness and usability, though there remains room for enhancement in areas like accessibility, collaboration features, and more advanced analysis capabilities.

8. Appendices & References

Technical Glossary

Project Terminology

Term	Definition	Documentation Link
QUEST	**Q**ualitative **U**ntersuchung und **E**valuation von **S**prachmodellen **T**ool - A comprehensive platform for interactive testing, comparison, and evaluation of language models	README.md
Adapter Pattern	Software design pattern implemented in QUEST to connect to multiple LLM providers through a unified interface	README.md
LLM	Large Language Model - AI models capable of understanding and generating human-like text	OpenAI Documentation

Term	Definition	Documentation Link
System Prompt	Initial instructions provided to language models to guide their behavior and responses	OpenAI Documentation
Thinking Block	Isolated reasoning sections in model responses that show the model's thought process	Evaluation.md

Evaluation Metrics

Term	Definition	Documentation Link
Cosine Similarity	Vector-based metric measuring the angular similarity between text embeddings, ranging from -1 (opposite) to 1 (identical)	Evaluation.md
Levenshtein Distance	Edit distance metric counting the minimum number of single-character operations needed to transform one text into another	Evaluation.md
Jaccard Similarity	Set-based similarity metric comparing word-level overlap between responses (0 = completely different, 1 = identical)	Evaluation.md
Embeddings	Vector representations of text used for semantic analysis and comparison	README.md

Frontend Technologies

Term	Definition	Documentation Link
React	Component-based JavaScript library for building user interfaces	React Documentation
TypeScript	Typed superset of JavaScript that compiles to plain JavaScript	TypeScript Documentation
Vite	Frontend build tool and development server with hot module replacement	Vite Documentation
Mantine UI	React components library with a focus on usability and customization	Mantine Documentation
ApexCharts	JavaScript charting library used for data visualization in the evaluation system	ApexCharts Documentation

Term	Definition	Documentation Link
Socket.IO Client	Client library for real-time, bidirectional communication with the server	Socket.IO Client Documentation
ReactMarkdown	Markdown renderer for React applications	react-markdown Documentation

Backend Technologies

Term	Definition	Documentation Link
Node.js	JavaScript runtime for server-side applications	Node.js Documentation
Express.js	Web application framework for Node.js	Express Documentation
Socket.IO Server	Server-side library for real-time, bidirectional communication	Socket.IO Server Documentation
JWT	JSON Web Tokens - compact, URL-safe means of representing claims securely between parties	JWT Introduction
bcrypt-ts	Password hashing and verification library for TypeScript	bcrypt-ts npm

LLM Providers

Term	Definition	Documentation Link
OpenAI	Cloud-based provider of language models like GPT-3.5 and GPT-4	OpenAI API Documentation
Ollama	Framework for running large language models locally	Ollama Documentation
Model Filter	Regular expression patterns used to filter available models from providers	README.md

Deployment Technologies

Term	Definition	Documentation Link
Docker	Platform for developing, shipping, and running applications in containers	Docker Documentation
Docker Compose	Tool for defining and running multi-container Docker applications	Docker Compose Documentation

Term	Definition	Documentation Link
Nginx	Web server used as a reverse proxy in the QUEST deployment	Nginx Documentation

Related Research and Resources

Reference	Type	Description	URL/DOI
KI-Med Collaboration Platform (KiMeKo)	Project	Parent project for QUEST, funded by BMBF (funding code 01IS24056D)	https://kimeko.digital-hub.sh/
University of Rostock	Institution	Academic institution behind the development of QUEST	https://www.uni-rostock.de/
Visual Analytics Center (VAC)	Research Center	Affiliated research center at University of Rostock	https://vac.uni-rostock.de/
Documentation on Evaluation Metrics	Technical	Detailed explanation of similarity metrics used in QUEST	Evaluation.md
QUEST Configuration Guide	Documentation	Comprehensive guide to configuring the QUEST server	README.md
QUEST Client Developer FAQ	Documentation	Frequently asked questions for client development	ClientDeveloperFAQ.md
QUEST Server Developer FAQ	Documentation	Frequently asked questions for server development	ServerDeveloperFAQ.md
Python Client Documentation	Documentation	Guide for using the Python client library for QUEST	README.md

Library Versions

QUEST uses the following key library versions (based on `package.json` files):

Client Libraries

- React: ^18.3.1
- react-apexcharts: ^1.7.0
- react-markdown: ^9.0.1
- socket.io-client: ^4.0.0
- TypeScript: ^5.0.0 (dev dependency)
- Vite: ^5.0.0 (dev dependency)

Server Libraries

- express: ^4.18.2
- jsonwebtoken: ^9.0.2
- openai: ^4.54.0
- socket.io: ^4.8.1
- TypeScript: ^5.3.3 (dev dependency)

Additional Resources

- Creative Commons License Information - QUEST is licensed under CC BY-NC-SA 4.0
- Docker Hub Repository - Pre-built Docker image for QUEST
- BMBF Funding Information - German Federal Ministry of Education and Research

Citation Format

To cite QUEST in academic work:

```
Gratzkowski, F., Bader, S., & Nicolay, R. (2024). QUEST:
Qualitative Untersuchung und
Evaluation von Sprachmodellen Tool [Software]. University of
Rostock.
https://git.informatik.uni-rostock.de/MMIS-KIMEKO/LLMApps
```